6.4 API Documentation

The last level of documentation we want to introduce in this chapter is API documentation and, specifically, the documentation of a web service's REST API. When developing web applications, you usually have a natural break between frontend and backend. The endpoints of the backend provide read and write interfaces for the frontend. Frontend and backend are often implemented by different teams, sometimes even by different companies. For this reason, clear and proper communication is of great importance at this point.

The OpenAPI Specification, also known under its former name, Swagger, is widely used for documenting REST APIs. Depending on the platform used, you describe the interfaces of your application in a separate document in YAML Ain't Markup Language (YAML), JavaScript Object Notation (JSON), or directly in the code as a comment or using a decorator.

6.4.1 Sample API Documentation

To create API documentation, we use an existing TypeScript application based on the Express framework. There is an extension for Express available called `swagger-ui-express`, which provides a graphical interface with the API documentation via a given path. In our example, the extension reads its configuration from a Swagger file, which is initially still empty, and makes the documentation available via the `/api` path.

The application has an endpoint, namely the `/users` path, which is used to deliver user data in JSON format. The data originates from a `UserService` class and represents instances of the `User` class. The code of the initial file of the application looks as follows:

```
import { readFileSync } from "node:fs";
import express from "express";
```

```
import swaggerUi from "swagger-ui-express";
import * as YAML from "yaml";
import UserService from "./user.service.js";

const app = express();

const yamlSpec = readFileSync("./swagger.yaml",
"utf8");
const swaggerDocument = YAML.parse(yamlSpec);

app.use(
  "/api",
  swaggerUi.serve,
  swaggerUi.setup(swaggerDocument)
);

const userService = new UserService();
app.get("/users", (request, response) => {
  response.json(userService.getAll());
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

In this case, we use the GitHub Copilot extension for Visual Studio (VS Code) to generate the API documentation. When you open the `swagger.yaml` file in your development environment, you can use the chat feature to issue the following prompt:

Prompt: Please create an API documentation for the express application in `#file: index.ts` using the user class in `#file:user.ts`. Please include example data.

> Certainly! Here's an example of how you can create an API documentation using Swagger YAML in the `swagger.yaml` file:
>
> ```
> openapi: 3.0.0
> info:
>   title: User API
>   version: 1.0.0
> ```

```
servers:
  - url: http://localhost:3000
paths:
  /api/users:
    get:
      summary: Get all users
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/User"
components:
  schemas:
    User:
      type: object
      properties:
        name:
          type: string
          example: John Doe
        email:
          type: string
          example: johndoe@example.com
        password:
          type: string
          example: password123
```

The context of API documentation is much more extensive than that of a simple function or class. The API is the entry point for client-server communication. The associated source code comprises not just a single file, but usually multiple files. In the example, these are the initial `index.ts` file and the `user.ts` file with the definition of the structure. You must make these files available to the AI tool to obtain a result that makes sense.

Apart from one small error, Copilot delivers exactly the result we expect. You only need to change the `/api/users` path to `/users`. Other than that, both the general information and the documentation for the endpoint and the data structure with the required sample data are correct. If you apply the

code and reload your application, you can access the Swagger UI via *http://localhost:3000/api* and test your endpoint from there.

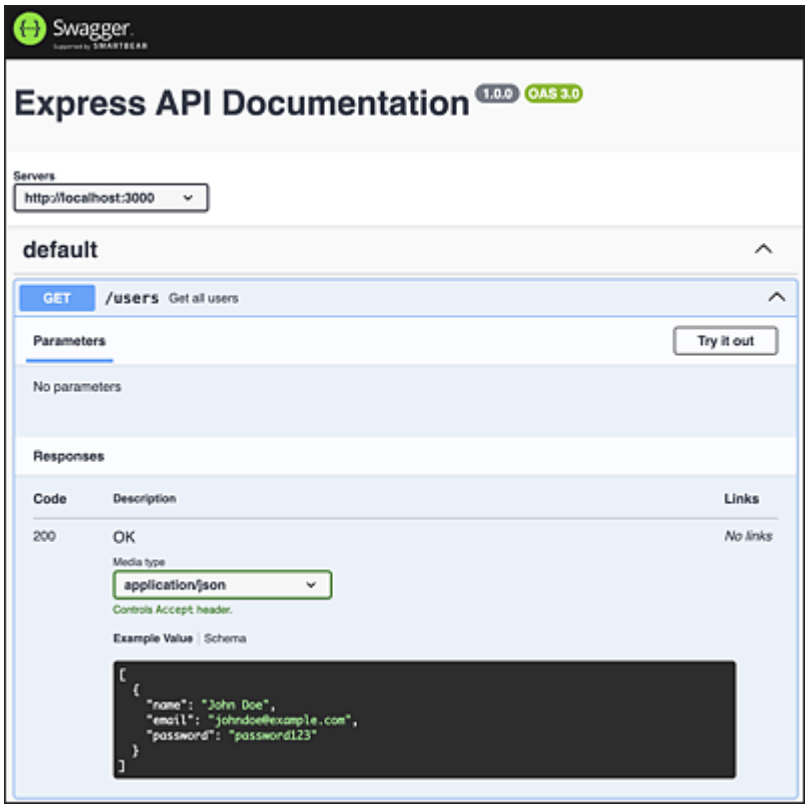

Figure 6.1 View of the Swagger UI

Click the **Try it out** button, and then **Execute** to send the request to the server and test your endpoint. Due to the configuration, the graphical interface not only provides information about the path and HTTP method but also about the structure of the response and the associated sample data.